

Mô hình máy và độ phức tạp (Cost Models and Complexity)

PHỤ THUỘC A/01 cho vai trò của chương này trong bước 2 của Pólya. Phần giải hệ thức truy hồi dùng vài công cụ tổng ở A/05, nhưng đọc được trước.

TÓM TẮT

Chương này cấp cái thước thứ nhất: *ý tưởng của tôi có đủ nhanh không*. Nó bắt đầu từ mô hình máy mà mọi phân tích ngầm giả định, đi qua ký hiệu tiệm cận, rồi tới ba công cụ dùng hằng ngày — bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, cách giải hệ thức truy hồi, và phân tích khấu hao. Phần cuối phá bỏ sự ngây thơ của mô hình: hằng số, bộ nhớ và cache có thể lật ngược kết quả tiệm cận ở kích thước thật. Nếu chỉ nhớ một điều: độ phức tạp không phải để khoe mà để *loại bỏ ý tưởng trước khi gõ code*, và bảng ở mục 3 là công cụ đáng thuộc nhất chương.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình RAM (random access machine)** — mô hình máy giả định: mỗi phép toán số học, so sánh, gán, và truy cập một ô nhớ bất kỳ đều mất một đơn vị thời gian. Mọi ký hiệu $O(\cdot)$ trong sách vở đều ngầm nói về mô hình này.
- **kích thước dữ liệu vào n** — đại lượng mà ta đo chi phí theo nó. Phải nói rõ n là gì — số phần tử, số đỉnh, số bit — vì cùng một thuật toán có thể là đa thức theo cách đo này và mũ theo cách đo kia.
- **trường hợp xấu nhất / trung bình** — chi phí lớn nhất trên mọi dữ liệu vào cỡ n , so với chi phí trung bình theo một phân phối giả định. Nói “trung bình” mà không nói phân phối là nói vô nghĩa.
- **hệ thức truy hồi (recurrence)** — phương trình kiểu $T(n) = 2T(n/2) + n$, mô tả chi phí của thuật toán đệ quy theo chi phí của chính nó trên dữ liệu nhỏ hơn.
- **khấu hao (amortized)** — chi phí trung bình mỗi thao tác trong một *dãy* thao tác xấu nhất. Khác với trung bình xác suất: ở đây không có ngẫu nhiên nào cả.
- **phân cấp bộ nhớ** — thanh ghi, L1, L2, L3, RAM, SSD, đĩa — mỗi tầng chậm hơn tầng trên hàng chục tới hàng nghìn lần. Mô hình RAM giả vờ chúng như nhau, và đó là chỗ nó sai nhiều nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ta phân tích thuật toán trên một mô hình máy mà không máy thật nào giống, và vì sao điều đó vẫn hữu ích?
- Q2. Hai thuật toán cùng $O(n \log n)$. Trong thực tế một cái nhanh gấp năm lần cái kia. Ký hiệu tiệm cận đã che mất cái gì?
- Q3. Vì sao mảng động tăng gấp đôi kích thước lại cho chi phí thêm phần tử là $O(1)$ khấu hao, trong khi có những lần thêm tốn tới $O(n)$?
- Q4. Đề cho $n \leq 2 \cdot 10^5$ và thời gian 1 giây. Vì sao từ hai con số đó bạn suy ra được gần như toàn bộ dạng lời giải cần tìm?
- Q5. Khi nào một thuật toán $O(n^2)$ là lựa chọn *đúng* thay vì $O(n \log n)$?

Trước khi gõ dòng code đầu tiên, có một câu hỏi cần trả lời và nó chỉ tốn ba mươi giây: *ý tưởng này chạy trong bao lâu với dữ liệu lớn nhất mà đề cho phép?* Người mới bỏ qua câu hỏi này và trả giá bằng cách cài xong một lời giải $O(n^2)$ rồi mới phát hiện n lên tới một triệu. Người có kinh nghiệm trả lời nó trước, và nhờ vậy loại được ba phần tư số ý tưởng trong đầu mà không mất một phút gõ phím nào.

Chương này dựng công cụ cho câu hỏi đó. Nhưng cũng cần nói ngay giới hạn của nó: độ phức tạp tiệm cận là một mô hình, và mô hình nào cũng có chỗ nói dối. Nửa cuối chương dành cho những chỗ nói dối ấy — hằng số, bộ nhớ, cache — vì trong công việc thật, chúng thường quyết định nhiều hơn cả bậc tiệm cận.

1.1 Mô hình máy: giả định mà mọi phân tích đều ngầm dùng

Khi ta nói “thuật toán này chạy $O(n \log n)$ ”, ta đang nói về một cái máy tưởng tượng gọi là *mô hình RAM*: bộ nhớ là một dãy ô đánh số, truy cập ô nào cũng mất một đơn vị thời gian, và mỗi phép toán cơ bản trên một số cũng mất một đơn vị.

Mô hình này sai ở ít nhất ba chỗ. Truy cập bộ nhớ *không* đồng giá — đọc một ô đang nằm trong cache nhanh hơn đọc một ô phải lấy từ RAM cỡ hai bậc. Phép toán *không* đồng giá — phép chia chậm hơn phép cộng nhiều lần, và phép nhân số lớn thì chi phí phụ thuộc số bit. Và số nguyên trong máy thật có kích thước hữu hạn, nên “một phép cộng” chỉ đúng khi số vừa trong một từ máy.

Vậy vì sao vẫn dùng? Vì mô hình này giữ được thứ ta cần và bỏ đi thứ ta không kiểm soát được. Nó cho phép so sánh hai thuật toán mà không phụ thuộc ngôn ngữ, trình biên dịch hay đời máy, và nó dự đoán đúng điều quan trọng nhất: *khi dữ liệu lớn gấp đôi thì thời gian tăng bao nhiêu lần*. Với dữ liệu đủ lớn, bậc tiệm cận luôn thắng hằng số. Sai lầm không nằm ở việc dùng mô hình, mà ở việc quên rằng mình đang dùng mô hình — mục 6 sẽ nói về đúng những lúc phải nhớ ra.

CẠM BẤY

Một chỗ mà mô hình RAM đánh lừa thường xuyên trong lập trình thi đấu: coi số nguyên lớn tùy ý là “một đơn vị”. Nếu bài toán làm việc với số có hàng nghìn chữ số, mỗi phép cộng đã tốn $O(\text{số chữ số})$, và độ phức tạp thật khác hẳn con số bạn vừa tính. Tương tự với chuỗi: so sánh hai chuỗi *không* phải $O(1)$ mà là $O(\text{độ dài})$ — đây là lý do một lời giải dùng `map<string, int>` có thể chậm hơn dự đoán hẳn một bậc.

1.2 Ký hiệu tiệm cận, nói cho đúng

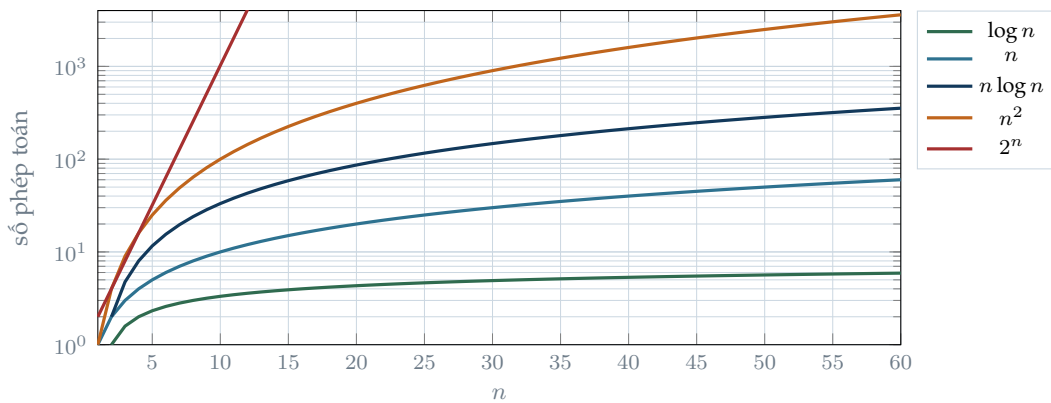
Ba ký hiệu cần phân biệt, và cách nhớ đơn giản nhất là coi chúng như dấu so sánh cho hàm số khi n lớn.

$O(f(n))$ là chặn trên: thuật toán chạy *không chậm hơn* $c \cdot f(n)$ với n đủ lớn. $\Omega(f(n))$ là chặn dưới: *không nhanh hơn*. $\Theta(f(n))$ là cả hai: đúng bậc đó. Trong thực tế người ta nói “thuật toán này $O(n \log n)$ ” và ý là $\Theta(n \log n)$; sự lỏng lẻo này vô hại, trừ khi bạn đang nói về *cận dưới của một bài toán* — lúc đó phải chính xác, vì “mọi thuật toán so sánh cần $\Omega(n \log n)$ ” là một phát biểu hoàn toàn khác với “thuật toán này chạy $O(n \log n)$ ”. Chương D/24 bàn kỹ.

Bậc tăng của các hàm hay gặp, xếp từ chậm tới nhanh:

$$1 \prec \log^* n \prec \log \log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n.$$

Điều đáng nhớ hơn danh sách này là *khoảng cách* giữa các bậc, và hình 1.1 cho thấy nó bằng hình.



Hình 1.1. Các bậc tăng trên trục dọc lôgarit. Hai điều đáng nhìn kỹ: n và $n \log n$ gần như song song — khoảng cách giữa chúng nhỏ tới mức trong thực tế hiếm khi đáng đánh đổi tính đơn giản để bỏ thừa số log; còn 2^n cất lên phía trên mọi đường khác trước khi n kịp đạt 20. Đó là toàn bộ lý do vì sao giới hạn $n \leq 20$ trên đề bài lại là một gợi ý mạnh.

Ba quy tắc rút gọn dùng hằng ngày: bỏ hằng số nhân ($3n^2 \rightarrow n^2$); chỉ giữ số hạng lớn nhất ($n^2 + n \log n \rightarrow n^2$); và với vòng lặp lồng nhau thì nhân, với các đoạn nối tiếp thì lấy cái lớn nhất. Riêng cơ sở của lôgarit thì bỏ được, vì đổi cơ sở chỉ là nhân với hằng số.

1.3 Từ giới hạn đề bài suy ra độ phức tạp cho phép

Đây là mục thực dụng nhất chương. Một máy chấm hiện đại thực hiện cỡ 10^8 tới 10^9 phép toán đơn giản mỗi giây — con số thô, phụ thuộc ngôn ngữ và loại phép toán, nhưng đủ tốt để ước lượng. Lấy 10^8 làm mốc an toàn cho giới hạn một giây, ta có bảng sau. Bảng này nên thuộc lòng; nó là thứ biến dòng “ $n \leq \dots$ ” trên đề thành một gợi ý về dạng lời giải.

Giới hạn n	Độ phức tạp cho phép	Dạng lời giải thường gặp
$n \leq 11$	$O(n!)$	duyệt mọi hoán vị
$n \leq 20\text{--}25$	$O(2^n \cdot n)$	duyệt tập con, quy hoạch động bitmask, ghép nhau ở giữa
$n \leq 100$	$O(n^4)$	bốn vòng lặp, quy hoạch động bốn chiều
$n \leq 500$	$O(n^3)$	Floyd–Warshall, nhân ma trận, quy hoạch động khoảng
$n \leq 5000$	$O(n^2)$	quy hoạch động hai chiều, mọi cặp
$n \leq 10^5\text{--}10^6$	$O(n \log n)$	sắp xếp, cây chỉ số, chia để trị, tập hợp
$n \leq 10^7\text{--}10^8$	$O(n)$	một lượt quét, hai con trỏ, sàng
$n \leq 10^{18}$	$O(\log n)$ hoặc $O(1)$	nhị phân, lũy thừa nhanh, công thức đóng

Cách dùng bảng này ngược với cách người ta hay nghĩ. Bạn không dùng nó để kiểm tra lời giải *sau khi* nghĩ ra, mà để *định hướng* việc nghĩ. Thấy $n \leq 20$ thì đừng phí thời gian tìm lời giải đa thức tinh vi — người ra đề đã nói với bạn rằng lời giải là mũ. Thấy $n \leq 5000$ mà bạn đang cố nghĩ ra $O(n \log n)$ thì bạn đang làm khó mình vô ích; $O(n^2)$ được chấp nhận và thường đơn giản hơn nhiều.

Ba lưu ý làm bảng chính xác hơn. Thứ nhất, để ý *tổng* n qua các bộ test: đề ghi “tổng n không vượt quá $2 \cdot 10^5$ ” nghĩa là bạn được phép làm tuyến tính trên mỗi test mà không lo số test lớn. Thứ hai, hằng số của thao tác quan trọng: 10^8 phép cộng số nguyên thì nhanh, nhưng 10^8 lần chèn vào map thì chắc chắn quá giờ, vì mỗi thao tác đó tốn hàng chục tới hàng trăm lần hơn. Thứ

ba, bộ nhớ cũng có giới hạn: một mảng long long kích thước 10^8 cần 800 MB, vượt xa mức cho phép thông thường — nên đôi khi ràng buộc bộ nhớ mới là ràng buộc quyết định.

CẦN NHỚ

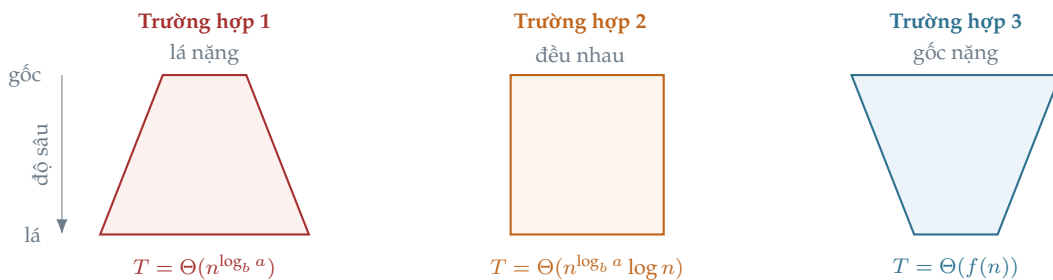
Đọc giới hạn *trước* khi nghĩ, không phải *sau*. Hai con số — n lớn nhất và giới hạn thời gian — thu hẹp không gian ý tưởng xuống còn một hoặc hai dạng. Đây là kỹ thuật rẻ nhất trong cả quyển sách này: tốn ba mươi giây, tiết kiệm hàng giờ.

1.4 Hệ thức truy hồi: chi phí của thuật toán đệ quy

Khi thuật toán gọi lại chính nó trên dữ liệu nhỏ hơn, chi phí được mô tả bằng hệ thức truy hồi. Có ba cách giải, xếp theo thứ tự nên thử.

Cách thứ nhất — vẽ cây đệ quy. Đây là cách trực giác nhất và hầu như luôn đủ. Vẽ cây các lời gọi, tính tổng chi phí ở mỗi tầng, rồi cộng các tầng lại. Với $T(n) = 2T(n/2) + n$: tầng gốc tốn n ; tầng hai có hai nút, mỗi nút tốn $n/2$, tổng vẫn là n ; mọi tầng đều tốn n ; cây sâu $\log_2 n$ tầng; vậy $T(n) = \Theta(n \log n)$. Toàn bộ merge sort nằm trong ba câu đó.

Cách thứ hai — định lý thợ (master theorem). Với dạng $T(n) = aT(n/b) + f(n)$, so sánh $f(n)$ với $n^{\log_b a}$: nếu f nhỏ hơn hẳn thì lá chiếm ưu thế và $T = \Theta(n^{\log_b a})$; nếu bằng nhau thì $T = \Theta(n^{\log_b a} \log n)$; nếu f lớn hơn hẳn (và thoả điều kiện đều đặn) thì gốc chiếm ưu thế và $T = \Theta(f(n))$. Hình 1.2 minh hoạ ba trường hợp bằng hình dáng cây.



Hình 1.2. Ba trường hợp của định lý thợ, nhìn theo “tầng nào tốn nhiều nhất”. Bề rộng mỗi tầng biểu diễn tổng chi phí của tầng đó. Nếu chi phí dồn về lá thì tổng do lá quyết định; nếu mọi tầng bằng nhau thì tổng bằng chi phí một tầng nhân số tầng; nếu dồn về gốc thì tổng do gốc quyết định. Nhớ hình này thì không cần nhớ công thức.

Cách thứ ba — đoán và chứng minh bằng quy nạp. Dùng khi hai cách trên không áp được, ví dụ với truy hồi không đều như $T(n) = T(n/5) + T(7n/10) + n$ của thuật toán chọn trung vị [1]. Đoán $T(n) \leq cn$, thay vào, và tìm c làm bất đẳng thức đúng: $cn/5 + 7cn/10 + n = 9cn/10 + n \leq cn$ khi $c \geq 10$. Vậy $T(n) = O(n)$, và cái đẹp của lời giải là ở chỗ tổng hai phân số nhỏ hơn 1.

Ba truy hồi đáng thuộc: $T(n) = T(n/2) + O(1)$ cho $\Theta(\log n)$ (tìm kiếm nhị phân); $T(n) = 2T(n/2) + O(n)$ cho $\Theta(n \log n)$ (merge sort); $T(n) = T(n-1) + O(n)$ cho $\Theta(n^2)$ (quicksort trường hợp xấu nhất).

1.5 Phân tích khấu hao: khi một thao tác đắt trả cho nhiều thao tác rẻ

Xét mảng động: khi đầy thì cấp phát mảng mới gấp đôi và sao chép toàn bộ. Một lần thêm phần tử có thể tốn $O(n)$. Nhưng nói “thêm phần tử là $O(n)$ ” thì sai lệch nghiêm trọng, vì lần đắt đó

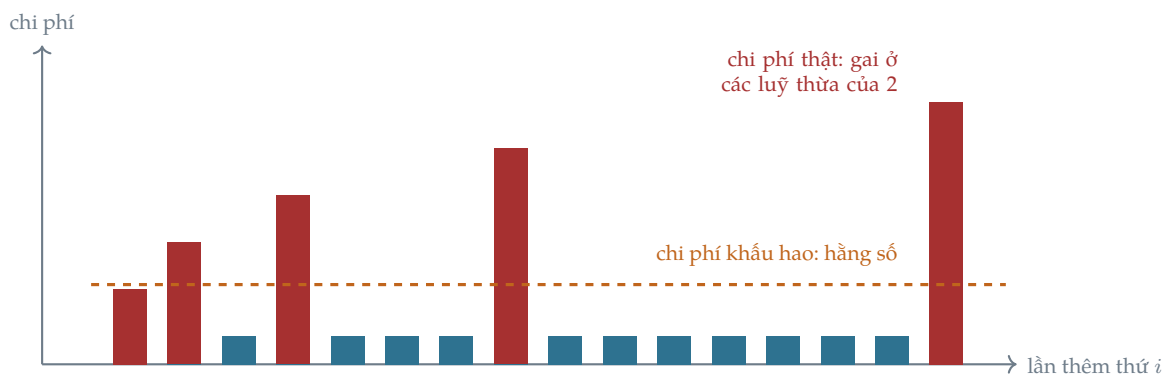
rất hiếm. Cách nói đúng: *chi phí khấu hao* của mỗi lần thêm là $O(1)$ — nghĩa là m lần thêm liên tiếp tốn tổng cộng $O(m)$, dù có vài lần lẻ tẻ tốn nhiều.

Chú ý một điểm bị nhầm rất nhiều: khấu hao *không* phải trung bình xác suất. Không có ngẫu nhiên nào ở đây; phát biểu đúng cho mọi dãy thao tác, kể cả dãy tệ nhất. Đây là công cụ Tarjan phát triển thành hệ thống[2], và nó có ba cách dùng.

Phương pháp tổng gộp: tính tổng chi phí của cả dãy rồi chia cho số thao tác. Với mảng động, tổng chi phí sao chép khi lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, nên trung bình mỗi lần thêm là $O(1)$. Chuỗi hình học ấy là toàn bộ lý do vì sao phải nhân đôi chứ không phải cộng thêm một hằng số — nếu mỗi lần chỉ nới thêm 10 ô, tổng chi phí sao chép thành $\Theta(n^2)$ và chi phí khấu hao thành $\Theta(n)$.

Phương pháp kế toán: gán cho mỗi thao tác một “giá vé” cố định cao hơn chi phí thật, phần dư gửi vào tài khoản, và những thao tác đắt rút từ tài khoản ra. Với mảng động: mỗi lần thêm trả 3 đơn vị — 1 cho việc ghi, 2 gửi tiết kiệm để sau này trả cho việc sao chép chính nó và một phần tử cũ. Nếu chứng minh được tài khoản không bao giờ âm thì giá vé chính là chi phí khấu hao.

Phương pháp thế năng: tổng quát hơn, gán cho cấu trúc một hàm thế năng Φ đo “độ lộn xộn tích lũy”; chi phí khấu hao của một thao tác bằng chi phí thật cộng độ tăng của Φ . Thao tác đắt được phép vì nó làm Φ giảm mạnh. Đây là cách dùng cho những cấu trúc tinh vi như splay tree[3] và Fibonacci heap[4].



Hình 1.3. Chi phí thật của thao tác thêm vào mảng động (cột) so với chi phí khấu hao (đường ngang). Các gai xuất hiện đúng ở lũy thừa của 2 và cao gấp đôi lần trước, nhưng cũng thưa gấp đôi — hai hiệu ứng triệt tiêu nhau, để lại tổng tuyến tính. Đây là hình đáng nhớ nhất về khấu hao: *gai cao hơn thì phải thưa hơn tương ứng*.

CẠM BẮY

Khấu hao chỉ hợp lệ khi bạn quan tâm tới *tổng* thời gian. Trong hệ thống thời gian thực — điều khiển robot, xử lý âm thanh — một thao tác đột ngột tốn $O(n)$ có thể làm hỏng hạn kỳ, dù chi phí khấu hao rất đẹp. Ở đó người ta chọn cấu trúc có đảm bảo *trường hợp xấu nhất* cho từng thao tác, chấp nhận hằng số lớn hơn. Chương E/26 nói kỹ tình huống này.

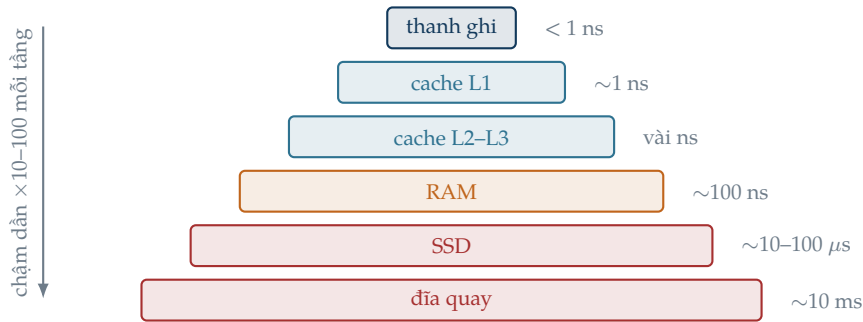
1.6 Chỗ mô hình nói dối: hằng số, bộ nhớ và cache

Đến đây ta có công cụ tiệm cận. Bây giờ là phần mà giáo trình hay nói lướt còn thực tế thì phạt rất nặng nếu bỏ qua.

Hằng số không nhỏ như ký hiệu gợi ý. Sắp xếp $O(n \log n)$ bằng `std::sort` nhanh hơn hẳn một *segment tree* $O(n \log n)$ tự viết, vì hằng số khác nhau cả chục lần. Ở kích thước nhỏ, thuật toán

“kém” thường thắng: đó là lý do các thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[5], [6].

Truy cập bộ nhớ mới là chi phí thật. Trên máy hiện đại, đọc một ô trong cache L1 nhanh hơn đọc từ RAM khoảng hai bậc độ lớn. Hệ quả rất cụ thể: duyệt một mảng liên tục nhanh hơn nhiều lần so với duyệt một danh sách liên kết cùng số phần tử, dù cả hai đều $O(n)$. Vì vậy mảng phẳng thường thắng cấu trúc con trỏ ở kích thước thật, và một segment tree cài bằng mảng nhanh hơn hẳn cài bằng con trỏ.



Hình 1.4. Phân cấp bộ nhớ với độ trễ theo bậc độ lớn (con số chỉ mang tính bậc, khác nhau theo máy). Mô hình RAM coi mọi tầng như nhau. Trong thực tế, một thuật toán “chậm hơn” về tiệm cận nhưng truy cập tuần tự có thể thắng một thuật toán “nhanh hơn” nhưng nhảy ngẫu nhiên khắp bộ nhớ — chương D/24 và E/26 khai thác đúng hiện tượng này.

Bộ nhớ cũng là một ngân sách. Một mảng hai chiều $10^4 \times 10^4$ kiểu int cần 400 MB — thường là vượt giới hạn. Khi gặp tình huống đó, hai lối thoát quen thuộc: giảm chiều của trạng thái quy hoạch động bằng cách chỉ giữ lại hàng trước (chương C/16), hoặc đổi sang cấu trúc thưa nếu dữ liệu thưa.

Và cuối cùng: đo, đừng đoán. Trong công việc thật, khoảng cách giữa dự đoán và thực tế đủ lớn để đáng bỏ hai phút chạy thử. Bài học của McSherry và cộng sự[7] rất đáng nhớ: nhiều hệ thống song song “có khả năng mở rộng” bị một chương trình đơn luồng viết cẩn thận đánh bại — vì chi phí điều phối lớn hơn phần được tăng tốc. Bậc tiệm cận không nói gì về những chi phí đó.

1.7 Đo bậc tăng trưởng thay vì đoán: phép thử nhân đôi

Cho tới đây ta luôn đi một chiều: từ mã nguồn suy ra chi phí. Chiều ngược lại — chạy chương trình rồi *đọc ngược* ra bậc tăng trưởng — rẻ hơn nhiều so với vẽ ngoài của nó và gần như không ai làm. Sedgewick và Wayne gọi kỹ thuật này là *phép thử tỉ lệ nhân đôi* (doubling ratio test), và khuyên chạy nó cho mọi chương trình mà hiệu năng có ý nghĩa[8].

Cách làm gồm ba bước. Một, viết một bộ sinh dữ liệu vào giống dữ liệu bạn thật sự gặp. Hai, chạy chương trình với n , rồi $2n$, rồi $4n$, ghi lại thời gian mỗi lần. Ba, lấy tỉ số giữa thời gian của hai lần liên tiếp. Nếu tỉ số ấy tiến tới một hằng số r , thì thời gian chạy xấp xỉ $a n^b$ với $b = \log_2 r$ — bậc của thuật toán hiện ra từ số đo, không cần đọc lại một dòng mã nào.

Vì sao tỉ số lại hội tụ? Chỉ cần một dòng tính. Nếu $T(n) \sim a n^b \lg n$ thì

$$\frac{T(2n)}{T(n)} = \frac{a(2n)^b \lg(2n)}{a n^b \lg n} = 2^b \left(1 + \frac{\lg 2}{\lg n}\right) \sim 2^b.$$

Nói cách khác, thừa số lôgarit — thứ luôn gây phiền khi phân tích bằng tay — gần như biến mất khỏi *tỉ số*. Đó là lý do phép đo này ổn định hơn ta tưởng, và cũng chính là lý do nó *không* phân biệt được $O(n)$ với $O(n \log n)$: cả hai đều cho tỉ số tiến về 2.

Ví dụ trong chính quyển sách của Sedgewick và Wayne đáng nhớ vì nó cho thấy sức mạnh dự đoán. Chương trình ThreeSum có ba vòng lặp lồng nhau; tỉ số đo được hội tụ về 8, tức $b = 3$, đúng bằng n^3 . Từ số đo 51,1 giây tại $n = 8000$, họ dự đoán 408,8 giây cho $n = 16\,000$ và 26 163 giây cho $n = 64\,000$ — chỉ bằng cách nhân liên tiếp với 8, không cần chạy thật.

Phép thử này trả lời ba câu hỏi mà lập luận trên giấy hay trả lời sai.

Chương trình có đúng như tôi nghĩ không? Nếu bạn tin mình viết $O(n \log n)$ mà tỉ số đo được là 4, thì ở đâu đó có một vòng lặp ẩn — thường là một phép sao chép chuỗi trong vòng lặp, một lần chèn vào giữa mảng, hoặc một lần tìm tuyến tính trong danh sách. Đây là cách rẻ nhất để bắt *lỗi hiệu năng*: loại lỗi không làm sai kết quả nên không phép kiểm tính đúng nào phát hiện được (E/25).

Chạy tới kích thước thật thì mất bao lâu? Nhân liên tiếp tỉ số cho một dự đoán đủ tốt, và nó biến câu hỏi “có kịp không” ở mục 3 từ ước lượng trong đầu thành một con số có cơ sở.

Ngưỡng hoà vốn nằm ở đâu? Khi hai lời giải khác bậc nhưng cũng khác hằng số, hai đường thời gian cắt nhau tại một điểm, và chỉ có đo mới tìm được điểm đó. Đây đúng là con số mà mọi thư viện sắp xếp phải chọn khi quyết định lúc nào chuyển sang sắp xếp chèn (C/14, E/26).

Ba điều kiện phải nhớ kèm, nếu không phép đo sẽ nói dối. Thứ nhất, tỉ số chỉ hội tụ với các bậc dạng lũy thừa; với thuật toán mũ nó tăng mãi — và chính việc *không* hội tụ đã là một câu trả lời. Thứ hai, bậc đo được là bậc *trên phân phối dữ liệu bạn sinh ra*: một bảng băm cho tỉ số 2 rất đẹp trên dữ liệu ngẫu nhiên vẫn có thể tệ đi n lần trên dữ liệu do đối thủ dựng (B/08). Thứ ba, ở kích thước nhỏ, thời gian bị chi phối bởi khởi động, cache lạnh và nhiễu đo; hãy bỏ vài điểm đầu và chỉ tin phần đuôi của bảng.

1.8 Gốc rễ: ký hiệu này mượn từ đâu và vì sao lại cần

Ký hiệu $O(\cdot)$ không sinh ra trong ngành máy tính. Nó xuất hiện cuối thế kỷ 19 trong lý thuyết số giải tích — Paul Bachmann dùng nó năm 1894, Edmund Landau phổ biến nó — để nói “sai số của xấp xỉ này không lớn hơn cỡ này” khi nghiên cứu phân bố số nguyên tố. Ở đó, biến chạy ra vô cùng là một đại lượng toán học thuần túy, và mối quan tâm là đáng điệu tiệm cận của hàm.

Ngành máy tính mượn ký hiệu ấy vì gặp đúng nhu cầu đó dưới hình thức khác: muốn so sánh hai thuật toán mà không phải nói “trên máy IBM 7090 với trình biên dịch này thì...”. Knuth là người chuẩn hoá cách dùng cho thuật toán và cũng là người kéo $\Omega(\cdot)$ và $\Theta(\cdot)$ vào để phân biệt chặn trên với chặn dưới — một phân biệt mà giới toán trước đó không cần tới, nhưng ngành thuật toán thì rất cần, vì ở đây ta phải nói được cả “thuật toán của tôi nhanh cỡ này” lẫn “mọi thuật toán đều không thể nhanh hơn cỡ này”.

Điều lý thú là mô hình máy cũng có gốc rễ tương tự: nó ra đời vì một mô hình cũ không dùng được cho mục đích mới. Máy Turing (1936) là mô hình chuẩn của lý thuyết tính toán, nhưng nó tính chi phí theo số bước trên băng, nên ngay cả việc đọc phần tử thứ i của mảng cũng tốn thời gian tỉ lệ i . Với câu hỏi “bài toán này có giải được không” thì chi tiết đó không quan trọng; với câu hỏi “thuật toán nào nhanh hơn” thì nó bóp méo mọi thứ. Mô hình RAM ra đời trong thập niên 1970 chính để lấp chỗ đó: cho phép truy cập ngẫu nhiên với giá một đơn vị, gần với máy thật hơn nhiều.

Còn từ “khấu hao” thì mượn thẳng từ kế toán, và nghĩa gốc soi sáng nghĩa kỹ thuật rất rõ. Trong kế toán, khi mua một cái máy đắt tiền dùng được mười năm, người ta không ghi toàn bộ chi phí vào tháng mua mà *phân bổ* nó ra suốt vòng đời. Tarjan mượn đúng ý đó: lần cấp phát lại tốn kém của mảng động không nên bị tính hết cho một thao tác thêm, mà nên phân bổ cho tất cả các thao tác đã hưởng lợi từ nó[2]. Khi đã thấy nghĩa gốc, phương pháp kế toán ở mục 5 không còn là mẹo mà là chính bút toán trả trước.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1894 — Bachmann, Landau	Cần nói gọn về sai số xấp xỉ trong lý thuyết số giải tích.	Ra đời ký hiệu O ; ban đầu không liên quan gì tới thuật toán.
1936 — Turing	Định nghĩa chính xác “tính được”.	Máy Turing: hoàn hảo cho câu hỏi <i>giải được hay không</i> , nhưng bóp méo chi phí truy cập nên không hợp để so tốc độ.
Thập niên 1960–70 — mô hình RAM	Cần một mô hình chi phí gần máy thật để phân tích và so sánh thuật toán.	Truy cập ngẫu nhiên giá một đơn vị; trở thành mô hình ngầm định của mọi ký hiệu $O(\cdot)$ trong sách vở.
1976 — Knuth	Trong thuật toán cần phân biệt “nhanh cỡ này” với “không thể nhanh hơn cỡ này”.	Chuẩn hoá bộ ba O, Ω, Θ ; mở đường cho lý thuyết cận dưới (Ω).
1985 — Tarjan	Các cấu trúc dữ liệu mới (splay tree, Fibonacci heap) có thao tác lẻ tẻ rất đắt nhưng tổng thể rất rẻ.	Hệ thống hoá phân tích khấu hao, mượn khái niệm phân bổ chi phí của kế toán.
1999 — Frigo và cộng sự	Khoảng cách tốc độ giữa CPU và bộ nhớ ngày càng lớn, mô hình RAM ngày càng nói dối.	Mô hình cache-oblivious: tính chi phí theo số lần chuyển khối bộ nhớ thay vì số phép toán (Ω).

I 1.9 Liên hệ ngang: so sánh với các cách đo chi phí khác

Mô hình RAM chỉ là một trong nhiều mô hình chi phí, và biết các mô hình khác giúp bạn nhận ra lúc nào mô hình quen thuộc đang nói dối. Bảng dưới đặt chúng cạnh nhau theo cùng một tiêu chí: đơn vị đếm, giả định chính, và chỗ hỏng.

Mô hình	Đếm cái gì	Hợp khi nào, hỏng khi nào
Máy Turing	Bước trên băng	Hợp cho câu hỏi giải được / độ khó lớp bài toán; hỏng khi so tốc độ vì truy cập không phải $O(1)$
RAM (mặc định)	Phép toán và truy cập	Hợp cho hầu hết phân tích; hỏng khi số rất lớn, khi chuỗi dài, khi cache quyết định
Độ phức tạp bit	Phép toán trên từng bit	Bắt buộc với số học số lớn, mật mã (Ω); nặng nề khi không cần
Bộ nhớ ngoài	Số lần đọc/ghi khối đĩa	Hợp với cơ sở dữ liệu, dữ liệu lớn hơn RAM; thừa khi mọi thứ nằm gọn trong bộ nhớ
Cache-oblivious	Số lần chuyển khối, không cần biết kích thước khối	Giải thích vì sao mảng thẳng con trở; khó phân tích hơn (Ω)
Song song (PRAM)	Số bước với p bộ xử lý	Hợp để tìm giới hạn song song hoá; bỏ qua chi phí đồng bộ và truyền tin, vốn thường mới là nút cổ chai
Luồng dữ liệu	Bộ nhớ dùng, số lượt quét	Hợp khi dữ liệu chảy qua một lần và không lưu được (Ω)
Năng lượng	Joule mỗi thao tác	Quan trọng trên thiết bị nhúng và robot; gần như vắng mặt trong sách giáo khoa thuật toán

Ngoài các mô hình tính toán, ý tưởng “chỉ quan tâm tốc độ tăng, bỏ hằng số” còn là một công cụ tư duy dùng khắp nơi ngoài ngành máy tính — và biết điều đó làm cho việc phân tích tiệm cận bớt giống một nghi thức học thuật.

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Phân tích thứ nguyên và luật tỉ lệ: bỏ hằng số, giữ số mũ	Cùng một động tác tư duy: muốn biết cái gì chi phối khi hệ lớn lên thì bỏ chi tiết và giữ số mũ. Khác: trong vật lý số mũ đến từ định luật bảo toàn; trong thuật toán nó đến từ cấu trúc đệ quy của lời giải.
Sinh học	Luật tỉ lệ trao đổi chất theo khối lượng cơ thể	Cùng dạng phát biểu “đại lượng này tăng theo lũy thừa của kích thước”. Bài học chung: khi hệ lớn gấp mười, thứ chi phối thường đổi hẳn — đúng như $O(n^2)$ thua $O(n \log n)$ ở quy mô lớn.
Kế toán, tài chính	Khấu hao tài sản, trả góp	Không phải chỉ giống mà là nguồn gốc trực tiếp của thuật ngữ: phân bổ một chi phí lớn cho nhiều kỳ hưởng lợi. Phương pháp kế toán ở mục 5 chính là bút toán trả trước.
Kinh tế	Chi phí biên, lợi thế quy mô	Chi phí khấu hao chính là chi phí biên trung bình của một thao tác. Cả hai đều nhắc: đừng đánh giá một hệ thống bằng chi phí của một sự kiện đơn lẻ.
Kỹ thuật hệ thống	Đo hiệu năng bằng hồ sơ chạy thật (profiling)	Bổ sung chứ không thay thế: tiệm cận nói đáng điều khi n lớn, profiling nói sự thật ở n hiện tại. Người chỉ dùng một trong hai đều sai — chi tiết ở E/26.
Ước lượng trạng thái, robot	Ngân sách tính toán theo chu kỳ thời gian thực	Ở đây cái quan trọng là <i>trường hợp xấu nhất mỗi khung hình</i> , không phải trung bình — nên cấu trúc có chi phí khấu hao đẹp có thể lại là lựa chọn sai. Xem cạm bẫy ở mục 5.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mô hình giữ lại thứ ta cần và bỏ đi thứ ta không kiểm soát. Điều ta cần là *tốc độ tăng khi dữ liệu lớn lên* — thứ quyết định lời giải có sống nổi ở giới hạn của đề hay không — và điều đó không phụ thuộc máy, ngôn ngữ hay trình biên dịch. Cái giá phải trả là mô hình mù với hằng số và với phân cấp bộ nhớ, nên nó chỉ đúng khi n đủ lớn; ở n nhỏ nó có thể sai hoàn toàn.
- A2. Ký hiệu tiệm cận cố ý bỏ hằng số nhân và các số hạng bậc thấp. Chênh lệch năm lần thường đến từ ba nguồn: hằng số của thao tác cơ bản (so sánh số nguyên rẻ hơn nhiều so với so sánh chuỗi hoặc phép chia), kiểu truy cập bộ nhớ (tuần tự so với nhảy ngẫu nhiên), và chi phí cấp phát. Với dữ liệu đủ lớn thì bậc luôn thắng; nhưng “đủ lớn” đôi khi lớn hơn dữ liệu thật của bạn.
- A3. Vì các lần đắt vừa cao hơn vừa thưa hơn theo đúng cùng một tỉ lệ, và hai hiệu ứng triệt tiêu nhau. Chi phí sao chép khi mảng lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, một cấp số nhân bị chặn bởi hai lần số hạng cuối. Chia cho n lần thêm ta được hằng số. Điều then chốt là nhân đôi: nếu chỉ nói thêm một lượng cố định thì tổng chi phí thành $\Theta(n^2)$ và khấu hao thành $\Theta(n)$.
- A4. Vì hai con số đó xác định số phép toán bạn được tiêu — cỡ 10^8 — và từ đó ra bậc cho phép, còn bậc cho phép thì gần như xác định họ kỹ thuật. Với $n \leq 2 \cdot 10^5$ và một giây, bậc cho phép là $O(n \log n)$: điều này loại mọi ý tưởng “xét mọi cặp”, và trở thẳng tới sắp xếp, hai con trỏ, cây chỉ số, chia để trị hoặc tập hợp có thứ tự. Bạn vừa cắt bỏ phần lớn không gian tìm kiếm ý tưởng chỉ bằng cách đọc một dòng của đề.
- A5. Ít nhất bốn tình huống. Khi n nhỏ và luôn nhỏ — hằng số nhỏ và code đơn giản thắng. Khi $O(n^2)$ truy cập tuần tự còn $O(n \log n)$ nhảy ngẫu nhiên — cache có thể lật ngược kết quả. Khi thuật toán phức tạp có nguy cơ sai — một lời giải đúng chậm hơn vẫn hơn một lời giải nhanh mà sai. Và khi thời gian của bạn là tài nguyên khan hiếm hơn thời gian máy, ví dụ trong kỳ thi còn 15 phút hoặc trong đoạn mã chạy mỗi tháng một lần.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới việc dọn một thư viện. Nếu có gấp đôi số sách thì bạn mất gấp đôi thời gian để xếp chúng lên kệ, nhưng chỉ mất thêm một chút thời gian để *tìm* một quyển nếu kệ đã sắp theo thứ tự — tìm kiếm nhị phân là thế. Còn nếu bạn phải so từng cặp sách với nhau thì số việc tăng gấp bốn

khi số sách tăng gấp đôi; đó là n^2 , và nó là lý do những việc kiểu “xét mọi cặp” hỏng rất nhanh khi dữ liệu lớn.

Ký hiệu $O(\cdot)$ chỉ là cách nói “khi mọi thứ lớn gấp đôi thì công việc tăng bao nhiêu lần”. Nó cố tình không quan tâm bạn xếp sách nhanh hay chậm, vì cái đó đổi theo người, còn tỉ lệ tăng thì không. Khấu hao là chuyện khác và cũng rất đời thường. Bạn không đi chợ mỗi lần cần một quả trứng; bạn đi một chuyến và mua cả vỉ. Chuyến đi đó tốn thời gian, nhưng chia cho số quả trứng thì rẻ. Mảng động cũng vậy: thỉnh thoảng nó dọn nhà một lần thật tốn, nhưng vì mỗi lần dọn nó tăng gấp đôi sức chứa nên các lần dọn thừa dần đúng bằng mức chúng đất lên.

Cuối cùng, cái bảng “ n tới đâu thì được phép chậm tới đâu” chỉ là câu hỏi: tôi có ngân sách khoảng một trăm triệu việc nhỏ, dữ liệu chừng này, vậy mỗi phần tử tôi được làm bao nhiêu việc?

Một câu: Độ phức tạp là để loại bỏ ý tưởng trước khi gõ code, không phải để trang trí lời giải sau khi xong.

Chỗ không rút gọn được: Ranh giới “ n đủ lớn để bậc tiệm cận thắng hằng số” không có công thức chung; chỉ đo trên máy thật mới biết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Mô hình RAM giả định gì	Mỗi phép toán cơ bản và mỗi truy cập ô nhớ bất kỳ tốn một đơn vị thời gian. Sai ở ba chỗ: cache, chi phí phép toán khác nhau, và số nguyên có kích thước hữu hạn.
••• Bảng giới hạn $\rightarrow$ độ phức tạp	$n \leq 20$: 2^n ; $n \leq 500$: n^3 ; $n \leq 5000$: n^2 ; $n \leq 10^5$: $n \log n$; $n \leq 10^7$: n ; $n \leq 10^{18}$: $\log n$. Ngân sách khoảng 10^8 phép toán mỗi giây.
••• $O()$, $\Omega()$, $\Theta()$ khác nhau	Chặn trên, chặn dưới, và cả hai. Nói lỏng $O()$ thay cho $\Theta()$ thì vô hại, trừ khi đang phát biểu cận dưới của một bài toán.
••• Ba truy hồi phải thuộc	$T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n)$; $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$; $T(n) = T(n-1) + O(n) \Rightarrow \Theta(n^2)$.
••• Khấu hao khác trung bình xác suất thế nào	Khấu hao không có ngẫu nhiên: nó là trung bình trên một dãy thao tác xấu nhất, đúng cho mọi dãy. Trung bình xác suất cần một phân phối giả định.
••• Vì sao mảng động phải nhân đôi	Vì tổng chi phí sao chép $1 + 2 + 4 + \dots + n < 2n$ là cấp số nhân, cho khấu hao $O(1)$. Nếu nói thêm hằng số thì tổng thành $\Theta(n^2)$.
•• Ba trường hợp của định lý thợ	So $f(n)$ với $n^{\log_b a}$: lá nặng $\Rightarrow \Theta(n^{\log_b a})$; đều nhau $\Rightarrow$ nhân thêm $\log n$; gốc nặng $\Rightarrow \Theta(f(n))$.
•• Ba cách phân tích khấu hao	Tổng gộp (cộng cả dây rồi chia); kế toán (giá vé cố định, gửi phần dư vào tài khoản); thế năng (hàm Φ đo độ lộn xộn tích lũy).
•• Khi nào khấu hao không dùng được	Hệ thời gian thực có hạn kỳ cho từng thao tác; ở đó cần đảm bảo trường hợp xấu nhất, chấp nhận hằng số lớn hơn.
•• Bẫy “một phép toán là $O(1)$ ”	Sai với số lớn tùy ý (chi phí theo số chữ số) và với chuỗi (so sánh tốn theo độ dài) — <code>map<string, int></code> chậm hơn dự đoán một bậc vì lý do này.
•• Vì sao mảng thẳng danh sách liên kết	Truy cập tuần tự thân thiện với cache; RAM chậm hơn L1 khoảng hai bậc. Cùng $O(n)$ nhưng khác nhau nhiều lần trên máy thật.
•• “Tổng n qua các test bị chặn” nghĩa là gì	Bạn được phép chạy tuyến tính trên mỗi test mà không lo số test lớn — đây là gợi ý rất mạnh về dạng lời giải.
• Bài học COST[7]	Nhiều hệ thống song song bị một chương trình đơn luồng viết tốt đánh bại, vì chi phí điều phối không xuất hiện trong phân tích tiệm cận.
•• Phép thử tỉ lệ nhân đôi	Chạy với $n, 2n, 4n, \dots$; nếu tỉ số thời gian tiến tới r thì bậc là $n^{\log_2 r}$. Không phân biệt được $O(n)$ với $O(n \log n)$, và chỉ đúng cho phân phối dữ liệu bạn tự sinh.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao nói “thuật toán này có độ phức tạp trung bình $O(n \log n)$ ” mà không nói gì thêm là chưa đủ?

| Vì trung bình luôn phải kèm phân phối. Quicksort có trung bình $\Theta(n \log n)$ với giả định thứ tự đầu vào ngẫu nhiên đều; nếu dữ liệu đến đã sắp sẵn và bạn chọn chốt là phần tử đầu, nó tụt xuống $\Theta(n^2)$. Trong hệ thống thật, dữ liệu hiếm khi ngẫu nhiên và đôi khi do kẻ tấn công chọn. Cách sửa đúng là tự tạo ra ngẫu nhiên — chọn chốt ngẫu nhiên — để đảm bảo không phụ thuộc phân phối đầu vào; chương C/20 bàn kỹ.

2. Tôi có lời giải $O(n\sqrt{n})$ và một lời giải $O(n \log^2 n)$. Chọn cái nào?

| Tính thử ở n thật. Với $n = 10^5$: $n\sqrt{n} \approx 3 \cdot 10^7$, còn $n \log^2 n \approx 10^5 \cdot 17^2 \approx 3 \cdot 10^7$ — gần như bằng nhau, nên yếu tố quyết định là hằng số và độ khó cài đặt. Với $n = 10^6$ thì $n\sqrt{n} = 10^9$ đã quá lớn còn $n \log^2 n \approx 4 \cdot 10^8$ vẫn có cửa. Bài học chung: khi hai bậc gần nhau, đừng so ký hiệu — thay số thật vào.

3. Định lý thợ không áp được cho truy hồi của tôi. Làm sao?

| Vẽ cây đệ quy, đó là công cụ tổng quát hơn và gần như luôn cho câu trả lời. Tính chi phí mỗi tầng, xem chi phí tăng, giảm hay giữ nguyên theo tầng. Tăng theo cấp số nhân thì tổng do lá quyết định; giảm theo cấp số nhân thì do gốc quyết định; giữ nguyên thì nhân với số tầng. Nếu cây không đều (như truy hồi của thuật toán chọn trung vị), đoán nghiệm rồi chứng minh bằng quy nạp.

4. Vì sao log trong độ phức tạp không cần ghi cơ số?

| Vì đổi cơ số chỉ nhân với hằng số: $\log_2 n = \log_{10} n / \log_{10} 2$. Ký hiệu tiệm cận bỏ hằng số nhân, nên $\Theta(\log_2 n) = \Theta(\log_{10} n)$. Nhưng khi ước lượng số phép toán thật thì cơ số rất quan trọng: $\log_2 10^6 \approx 20$ còn $\log_{10} 10^6 = 6$, lệch hơn ba lần.

5. “Độ phức tạp giả đa thức” nghĩa là gì và vì sao nó quan trọng?

| Là độ phức tạp đa thức theo giá trị của số trong dữ liệu vào, chứ không theo số bit biểu diễn chúng. Quy hoạch động cho bài xếp ba lô chạy $O(nW)$ với W là sức chứa — nhìn thì đa thức, nhưng W được ghi bằng $\log W$ bit, nên theo kích thước dữ liệu vào thật thì đó là mũ. Đây chính là lý do bài ba lô vẫn là NP-khó dù có lời giải quy hoạch động; chương D/23 nói kỹ.

6. Tôi nên tính độ phức tạp bộ nhớ thế nào cho nhanh?

| Đếm mảng lớn nhất rồi nhân với kích thước phần tử: int 4 byte, long long và con trỏ 8 byte. Nhớ rằng các cấu trúc có con trỏ tồn thêm nhiều: mỗi nút của map hoặc set thường tốn cỡ 48 byte cho một phần tử vài byte. Đó là lý do một set<int> với 10^7 phần tử là không khả thi trong khi mảng int cùng kích thước chỉ tốn 40 MB.

7. Nếu thuật toán của tôi vượt giới hạn thời gian, tôi nên tìm thuật toán tốt hơn hay tối ưu hằng số?

| Trả lời bằng khoảng cách. Nếu bạn chậm gấp 2–3 lần, tối ưu hằng số thường đủ: bỏ cấu trúc có con trỏ, đổi map sang mảng, tránh cấp phát trong vòng lặp, đọc dữ liệu nhanh hơn. Nếu chậm gấp 50 lần trở lên, không có tối ưu hằng số nào cứu được — bậc của bạn sai và phải đổi ý tưởng. Ước lượng bằng bảng ở mục 3 trước khi quyết định, để khỏi tối ưu vi mô một lời giải chết từ trong ý tưởng.

8. Phân tích trường hợp xấu nhất có bị quan quá không, khi dữ liệu thật hiếm khi tệ như vậy?

| Có và không. Trong lập trình thi đấu, dữ liệu được cố ý làm tệ nhất có thể, nên phân tích xấu nhất là phân tích duy nhất đáng tin. Trong hệ thống thật, phân tích xấu nhất đôi khi quá bị quan — nhưng nó lại là điều bạn quan tâm nhất khi có hạn kỳ hoặc khi kẻ tấn công có thể chọn dữ liệu (băm là ví dụ kinh điển, xem chương B/08). Cách dùng thực dụng: dùng trường hợp xấu nhất để đảm bảo an toàn, và dùng đo đạc thực nghiệm để tối ưu.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Với mỗi giới hạn $n = 10, 20, 500, 5000, 10^5, 10^7$, viết ra bậc cho phép và một thuật toán mẫu	Tự luyện	Mục tiêu là thuộc bảng mục 3 tới mức phản xạ; kiểm tra lại sau một tuần.
• Tính độ phức tạp của ba đoạn code có vòng lặp lồng nhau mà bước nhảy không đều	Bài tập kinh điển	Vòng lặp <code>for(i=1; i<=n; i*=2)</code> cho $\log n$; vòng lặp trong chạy tới i cho tổng $n^2/2$.
• Giải ba truy hồi cơ bản bằng cây đệ quy, không dùng định lý thợ	[9]	Vẽ cây thật ra giấy; mục tiêu là thấy được “tầng nào nặng” chứ không phải ra đáp số.
•• Chứng minh chi phí khấu hao của mảng động bằng cả ba phương pháp	[2]	Ba phương pháp cho cùng kết quả; điều cần rút ra là mỗi phương pháp dễ dùng ở tình huống khác nhau.
•• Phân tích chi phí khấu hao của bộ đếm nhị phân tăng dần	Bài tập kinh điển	Mỗi lần tăng lật trung bình 2 bit; dùng phương pháp kế toán với “một đơn vị gửi kèm mỗi bit 1”.
•• Đo thời gian thật: sắp xếp 10^6 số bằng <code>std::sort</code> và bằng cây tìm kiếm	Thực nghiệm	Cùng $O(n \log n)$ nhưng khác nhau nhiều lần; giải thích khác biệt bằng cache và cấp phát.
•• Duyệt mảng 10^7 phần tử theo thứ tự và theo hoán vị ngẫu nhiên, đo hai thời gian	Thực nghiệm	Đây là cách rẻ nhất để thấy phân cấp bộ nhớ bằng số liệu của chính máy bạn.
••• Giải truy hồi $T(n) = T(n/5) + T(7n/10) + n$ bằng quy nạp	[1]	Đoán $T(n) \leq cn$ và tìm c ; mấu chốt là $1/5 + 7/10 < 1$.
••• Tìm một trường hợp n nhỏ mà thuật toán $O(n^2)$ nhanh hơn $O(n \log n)$ trên máy bạn	Thực nghiệm	Sắp xếp chèn so với sắp xếp trộn; tìm điểm giao và giải thích vì sao thư viện thật cũng chuyển ngưỡng ở quanh đó.
••• Ước lượng bộ nhớ của quy hoạch động hai chiều $10^4 \times 10^4$ rồi tìm cách giảm chiều	Chương C/16	Chỉ giữ hàng trước; đây là kỹ thuật bạn sẽ dùng lại rất nhiều.
••• Đọc lại một lời giải cũ của bạn bị quá thời gian và phân loại: sai bậc hay sai hằng số	Nhật ký của bạn	Quy tắc ở mục hỏi đáp: chậm 2–3 lần thì sửa hằng số, chậm 50 lần thì phải đổi ý tưởng.

Kết chương và đọc tiếp

Chương này cho cái thước thứ nhất. Ba thứ đáng mang theo: bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, hình ảnh “tầng nào nặng” để giải truy hồi mà không cần nhớ công thức, và ý niệm khấu hao — gai cao thì phải thưa tương ứng. Cùng với đó là một sự dè dặt lành mạnh: tiệm cận là mô hình, và ở kích thước thật thì hằng số cùng cache có tiếng nói riêng.

Nhưng nhanh chưa đủ. Một lời giải $O(n \log n)$ sai thì tệ hơn một lời giải $O(n^2)$ đúng. Chương A/04 cấp cái thước thứ hai — công cụ để biết lời giải *đúng* — và bạn sẽ thấy công cụ ấy, bất biến, không chỉ dùng để chứng minh mà còn dùng để *nghĩ ra* thuật toán.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 1.4	Addison-Wesley, 2011	Nguồn của phép thử nhân đôi và của cách viết $\sim$ cho hằng số. Đọc mục “Doubling ratio experiments” — ba trang, dùng được ngay.
nền tảng	Tarjan, “Amortized Computational Complexity”	SIAM Journal on Algebraic and Discrete Methods, 1985	Bài báo hệ thống hoá ba phương pháp khấu hao. Đọc phần thể năng nếu muốn hiểu vì sao splay tree và Fibonacci heap phải phân tích kiểu đó.
tra khi cần	Bentley & McIlroy, “Engineering a Sort Function”	Software: Practice and Experience, 1993	Tài liệu tốt nhất về khoảng cách giữa “thuật toán trong sách” và “hàm dùng được”: ngưỡng hoà vốn, chọn chốt, dữ liệu ác ý.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Đọc khi muốn một mô hình chi phí có tính tới phân cấp bộ nhớ mà không phải biết kích thước cache ($D/24$).
tra khi cần	McSherry, Isard, Murray, “Scalability! But at What COST?”	HotOS, 2015	Sáu trang, đổi cách bạn đọc mọi báo cáo hiệu năng song song. Đọc bảng so sánh với chương trình đơn luồng.

Nguồn

- [1] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [2] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [3] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [4] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [5] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [6] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [7] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.